13L1-Logic

Generated by Scribe.AI

December 18, 2024

Slide 1

Content

CS 24300: AI Basics
Tony Bergstrom
Slides adapted from Yexiang Xue and Tony Bergstrom

Description

This slide is a title slide for a lecture on Artificial Intelligence Basics, likely part of a course titled CS 24300. It displays the course title, "CS 24300: AI Basics," and the name of the instructor, "Tony Bergstrom." The bottom line indicates that the slides are adapted from other sources, specifically Yexiang Xue and Tony Bergstrom. This is a standard introductory slide format, providing essential course information at the beginning of a presentation.

Figures

Slide 2

Content

Logic and the Satisfiability Problem

Description

This slide displays a title for a section of a lecture on Artificial Intelligence Basics. The title is "Logic and the Satisfiability Problem." It's likely the beginning of a section or module dedicated to exploring logical reasoning and the computational problem of determining whether a given logical formula has a solution (is satisfiable). The context of the course suggests that this topic will be discussed in the context of logical reasoning, problem-solving, and potentially, algorithms for finding solutions to such problems.

Figures

Slide 3

Content

Intelligent Systems Integrate Learning and Reasoning

[rectangle, draw, fill=teal!60] (data) at (0,0) Data; [rectangle, draw, fill=teal!60] (knowledge) at (4,0) Knowledge; [rectangle, draw, fill=teal!60] (action) at (8,0) Action;

[below of=data, node distance=0.5cm] (learning) Learning; [below of=knowledge, node distance=0.5cm] (reasoning) Reasoning; [below of=action, node distance=0.5cm] (reaction) Reaction; [above of=learning, node distance=0.5cm] (perception) Perception; [above of=reasoning, node distance=0.5cm] (reaction2) Reaction; [above of=reaction, node distance=0.5cm] (action2) Action;

[->, teal!60] (data) – (knowledge); [->, teal!60] (knowledge) – (action);

[below=1cm of knowledge] (description) An intelligent agent is a system that perceives its environment and takes actions that maximize its chances of success.
– Russell Norvig, 2003
* Pictures from Disney.; [image, inner sep=0pt, width=2cm, height=2cm, right=0.5cm of learning] (robot) at (2, -1) ;

Description

This slide, part of a lecture on Artificial Intelligence Basics, presents a visual model of how intelligent systems integrate learning and reasoning. It depicts a flowchart-like diagram with three main components: Data, Knowledge, and Action. Arrows connect these components, illustrating the flow of information and processes. The Data component is connected to the Knowledge component via an arrow labeled "Learning," indicating that learning is the process that transforms data into knowledge. The Knowledge component is connected to the Action component via an arrow labeled "Reasoning," signifying that reasoning is the process that uses knowledge to determine actions. A small image of a robot is positioned next to the "Learning" label, visually representing the concept of intelligent agents. The slide also includes a textual description below the Knowledge component, defining an intelligent agent as a system that perceives its environment and takes actions to maximize its chances of success. This description is attributed to Russell Norvig (2003), highlighting the source of the concept. The inclusion of "Pictures from Disney" suggests that the image of the robot might be from a Disney production, possibly used for illustrative purposes. The overall structure of the slide emphasizes the cyclical nature of perception, learning, reasoning, and action in intelligent systems.

Figures

(a)

Slide 4

Content

The Quest for Machine Reasoning

Logic reasoning: a cornerstone of Artificial Intelligence

Objective: Develop foundations and technology to enable effective, practical, large-scale automated reasoning

H. Simon (1956): ... within ten years, computers would beat the world chess champion, compose "aesthetically satisfying" original music, and prove new mathematical theorems...

Achieved in 40 Years (not 10)

- 1997 the music composed by David Cope's programs cannot be distinguished from that composed by Mozart, Beethoven, and Bach.
- 1976 (simplified and accepted 1997), a computer was used in the proof of the long- unsolved "four color problem.
- 1996, deep blue beat Garry Kasparov

Herbert Simon Portrait: https://en.m.wikipedia.org/wiki/File:Herbert$_s imon_r ed_c omplete.jpg$

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the quest for machine reasoning. It highlights logic reasoning as a fundamental aspect of AI. The objective is to develop the foundations and technology for effective, practical, and large-scale automated reasoning. A quote from Herbert Simon (1956) is presented, outlining ambitious goals for computers within a decade, including beating the world chess champion, composing music, and proving mathematical theorems. The slide then lists achievements in the field over approximately 40 years, including specific milestones like David Cope's music composition programs (1997), the proof of the four-color theorem (simplified and accepted in 1997), and Deep Blue's victory over Garry Kasparov (1996). The slide concludes with a link to a portrait of Herbert Simon, likely used for context and recognition of the influential figure in AI.

Figures

Slide 5

Content

The Quest for Machine Reasoning

Machine Reasoning (1960-90s) Computational complexity of reasoning appeared to severely limit real-world applications

Current reasoning technology Revisiting the challenge: Significant progress with new ideas / tools for dealing with complexity (scale-up), uncertainty, and multi- agent reasoning

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the historical and current challenges in machine reasoning. The title, "The Quest for Machine Reasoning," sets the stage for a discussion about the evolution of automated reasoning in AI. The first section, "Machine Reasoning (1960-90s)," highlights a period where the computational complexity of reasoning was a significant obstacle to developing practical applications. The phrase "appeared to severely limit real-world applications" indicates that the limitations of early reasoning techniques hindered the development of useful AI systems. The second section, "Current reasoning technology," suggests a shift in focus and approach. The phrase "Revisiting the challenge" implies that researchers are addressing the limitations of the past. The phrase "Significant progress with new ideas / tools" indicates that new methods and technologies are being explored. The final part of the section, "complexity (scale-up), uncertainty, and multi-agent reasoning," lists key areas of focus in current research, suggesting that researchers are tackling the challenges of scaling up reasoning systems, handling uncertainty in data, and dealing with multiple interacting agents. The overall message is that while early attempts at machine reasoning faced significant limitations, current research is actively addressing these challenges with new approaches.

Figures

Slide 6

Content

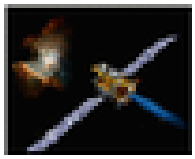
Description

This slide, part of a lecture on Artificial Intelligence Basics, presents a visual model of general automated reasoning. It depicts a flowchart-like diagram illustrating the process of using automated reasoning to solve problems. The diagram shows a sequence of steps: starting with a "Problem Instance," which is then processed by a "Model Generator (Encoder)." The output of the Model Generator is then fed into a "General Inference Engine," which produces a "Solution." The diagram distinguishes between "Domain Specific" and "Generic" components. The "Domain Specific" component is connected to the "Problem Instance" and represents problem instances specific to a particular domain (e.g., logistics, chess, planning, verification). The "Generic" component is connected to the "Model Generator (Encoder)" and represents a general model applicable to a range of domains. The slide also includes a description of the research objective: "Better reasoning and modeling technology," and an impact statement: "Faster solutions in several domains." The overall structure of the slide emphasizes the process of using general reasoning techniques to solve problems in various domains, potentially leading to faster solutions compared to domain-specific approaches.

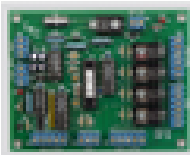
Figures

(a)

(b)



(c)



(d)

Slide 7

Content

Reasoning Can Be Defined Using Logic

Conclusion: a statement expressing the particular question being asked

Assumptions: a collection of statements expressing all the relevant information available to the program

Solving automated problems: proving the conclusion from the given assumptions by the systematic application of rules of deduction

E.g., prove mathematical theorems, decide winning move when playing games.

Logic is the language of automated reasoning.

Description

This slide, part of a lecture on Artificial Intelligence Basics, defines automated reasoning using logic. It describes the key components of a logical reasoning system: a **Conclusion**, which is the statement to be proven; **Assumptions**, which are the given statements; and the process of **Solving automated problems**, which involves deducing the conclusion from the assumptions using rules of deduction. The slide provides examples of tasks that can be solved using this approach, such as proving mathematical theorems and deciding winning moves in games. The final line emphasizes that logic is the fundamental language used in automated reasoning systems.

Figures

Slide 8

Content

Propositional logic (syntax)

Propositional logic is the simplest logic - illustrates basic ideas
The proposition symbols P_1, P_2 etc are sentences
If S is a sentence, $\neg S$ is a sentence (negation)
If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)
If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)
If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (implication)
If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (biconditional)

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces propositional logic syntax. It states that propositional logic is the simplest form of logic, used to illustrate fundamental ideas in logic. The slide defines propositional logic syntax by listing the components and rules for constructing logical statements. It explains that proposition symbols (like P_1, P_2, etc.) are considered basic sentences. The slide then defines how more complex sentences can be formed using logical connectives. Specifically, it defines negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), implication ($\Rightarrow$), and biconditional ($\Leftrightarrow$) as operations that combine simpler sentences to create more complex ones. Each line describes a rule for creating a new sentence from existing ones, along with the name of the logical connective used. The context of the course is crucial for understanding how propositional logic forms the basis for more complex reasoning systems in AI.

Figures

Slide 9

Content

Propositional logic (Semantics)

Each model specifies true/false for each proposition symbol

E.g.

S1	S2	S3
false	true	false

With these symbols, 8 possible models, can be enumerated automatically. Rules for evaluating:
¬S is true iff S is false
$S_1 \wedge S_2$ is true iff S_1 is true and S_2 is true
$S_1 \vee S_2$ is true iff S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$ is true iff S_1 is false or S_2 is true
i.e., $S_1 \Rightarrow S_2$ is false iff S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$ is true iff $S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true
Simple recursive process evaluates an arbitrary sentence, e.g.,
$\neg S_1 \wedge (S_2 \vee S_3) = \text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the semantics of propositional logic. It defines how truth values are assigned to propositional symbols (S_1, S_2, S_3, etc.) to create models. An example is given, showing that S_1 is false, S_2 is true, and S_3 is false. The slide then presents rules for evaluating compound propositions using logical connectives: negation (¬), conjunction (∧), disjunction (∨), implication (⇒), and biconditional (⇔). Each rule specifies the conditions under which a compound proposition is true or false in terms of the truth values of its constituent propositions. The slide emphasizes that these rules can be used to systematically evaluate any propositional sentence. The example at the end demonstrates how a complex proposition is evaluated using the rules, showing that $\neg S_1 \wedge (S_2 \vee S_3)$ evaluates to true when S_1 is false, S_2 is false, and S_3 is true. The context of the course is crucial for understanding how propositional logic forms the basis for more complex reasoning systems in AI.

Figures

Slide 10

Content

Truth Table

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Description

This slide presents a truth table for propositional logic. The table systematically evaluates the truth values of compound propositions (formed using logical connectives) based on the truth values of the constituent propositions P and Q. The columns represent the propositions P, Q, the negation of P ($\neg P$), the conjunction of P and Q ($P \wedge Q$), the disjunction of P and Q ($P \vee Q$), the implication of P to Q ($P \Rightarrow Q$), and the biconditional of P and Q ($P \Leftrightarrow Q$). Each row of the table shows the truth values of these compound propositions for all possible combinations of truth values for P and Q. The context of the course (Artificial Intelligence Basics) indicates that this slide is part of a lesson on propositional logic, which is a fundamental component of automated reasoning in AI.

Figures

(a)

Slide 11

Content

Let's Try it:

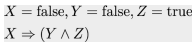
$X = \text{false}, Y = \text{false}, Z = \text{true}$

$X \Rightarrow (Y \wedge Z)$

Description

This slide, part of a lecture on Artificial Intelligence Basics, presents a propositional logic example. It shows a "Let's Try it" section, indicating an exercise or demonstration. The slide defines three propositional variables: X, Y, and Z, assigning them the truth values false, false, and true, respectively. The slide then presents a compound proposition: $X \Rightarrow (Y \wedge Z)$. The context of the course suggests that the slide is demonstrating the evaluation of a logical implication using the given truth values for the variables. The slide is likely part of a section on propositional logic semantics, showing how to determine the truth value of a complex statement based on the truth values of its components.

Figures



(a)

Slide 12

Content

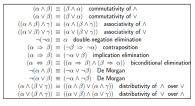
Logical Equivalence

- $(a \wedge \beta) \equiv (\beta \wedge a)$ commutativity of $\wedge$
- $(a \vee \beta) \equiv (\beta \vee a)$ commutativity of $\vee$
- $((a \wedge \beta) \wedge \gamma) \equiv (a \wedge (\beta \wedge \gamma))$ associativity of $\wedge$
- $((a \vee \beta) \vee \gamma) \equiv (a \vee (\beta \vee \gamma))$ associativity of $\vee$
- $(\neg\neg a) \equiv a$ double-negation
- $(a \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg a)$ contraposition
- $(a \Leftrightarrow \beta) \equiv ((a \Rightarrow \beta) \wedge (\beta \Rightarrow a))$ biconditional elimination
- $\neg(a \wedge \beta) \equiv (\neg a \vee \neg\beta)$ De Morgan
- $\neg(a \vee \beta) \equiv (\neg a \wedge \neg\beta)$ De Morgan
- $(a \wedge (\beta \vee \gamma)) \equiv ((a \wedge \beta) \vee (a \wedge \gamma))$ distributivity of $\wedge$ over $\vee$
- $(a \vee (\beta \wedge \gamma)) \equiv ((a \vee \beta) \wedge (a \vee \gamma))$ distributivity of $\vee$ over $\wedge$

Description

This slide presents a list of logical equivalences in propositional logic. Each line shows an equivalence between two propositional formulas, along with a label indicating the rule of inference used to derive the equivalence. The equivalences cover various logical connectives, including conjunction ($\wedge$), disjunction ($\vee$), implication ($\Rightarrow$), biconditional ($\Leftrightarrow$), and negation ($\neg$). The rules listed include commutativity, associativity, double negation, contraposition, De Morgan's laws, and distributivity. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a lesson on propositional logic, which is a fundamental component of automated reasoning in AI. The equivalences are crucial for simplifying and manipulating logical expressions.

Figures



(a)

Slide 13

Content

Validity and satisfiability

A sentence is valid if it is true in all models (aka tautologies) e.g., True, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

A sentence is satisfiable if it is true in some model, e.g., A

A sentence is unsatisfiable if it is true in no models, e.g., $A \wedge \neg A$

Description

This slide defines validity, satisfiability, and unsatisfiability of sentences in a logical system, likely in the context of propositional logic. It gives examples of each concept. "Validity" means a sentence is true in all possible models (or interpretations). "Satisfiability" means a sentence is true in at least one model. "Unsatisfiability" means a sentence is false in all models. The examples given illustrate these concepts with propositional logic expressions. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a lesson on knowledge representation and reasoning, where understanding these concepts is crucial for building logical systems and reasoning agents.

Figures

Slide 14

Content

Examples

$p \wedge (q \vee \neg p) \wedge (\neg q \vee \neg r)$ satisfiable
$p \wedge (q \vee \neg p) \wedge (\neg q \vee \neg p)$ unsatisfiable

Description

This slide presents two examples of propositional logic formulas and their classifications as either satisfiable or unsatisfiable. The first example, $p \wedge (q \vee \neg p) \wedge (\neg q \vee \neg r)$, is labeled as "satisfiable," indicating that there exists at least one assignment of truth values to the variables p, q, and r that makes the entire formula true. The second example, $p \wedge (q \vee \neg p) \wedge (\neg q \vee \neg p)$, is labeled as "unsatisfiable," meaning no assignment of truth values to the variables can make the formula true. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a lesson on propositional logic, specifically focusing on the evaluation of logical formulas and their properties. The examples are likely used to illustrate the concepts of satisfiability and unsatisfiability in propositional logic, which are fundamental to automated reasoning and knowledge representation in AI.

Figures

Slide 15

Content

The Satisfiability Problem

Let V be a finite set of Boolean valued variables and let ϕ denote a propositional formula on V. An assignment v on V is a satisfying assignment for ϕ if we have $v(\phi) = \text{true}$. The propositional formula ϕ is satisfiable if there exists a satisfying assignment for ϕ. Deciding whether or not a propositional formula is satisfiable is called the Boolean satisfiability problem, denoted by SAT.

Description

This slide introduces the Boolean Satisfiability Problem (SAT). It defines a satisfying assignment for a propositional formula ϕ over a set of Boolean variables V as an assignment of truth values to the variables in V that results in ϕ evaluating to true. A propositional formula is considered satisfiable if at least one such satisfying assignment exists. The slide then states that the problem of determining whether a given propositional formula is satisfiable is known as the Boolean satisfiability problem, and it is denoted by SAT. The context of the course (Artificial Intelligence Basics) indicates that this slide is part of a lesson on knowledge representation and reasoning, specifically focusing on the fundamental computational problem of determining the satisfiability of logical expressions.

Figures

Slide 16

Content

Conjunctive Normal Form (CNF)

A literal is either a propositional variable or the negation of a propositional variable. A clause is a disjunction of literals. A formula is in conjunctive normal form (CNF), if it is a conjunction of clauses; for instance:

$(p \vee \neg q \vee r) \wedge (q \vee r) \wedge (\neg p \vee \neg q) \wedge r$
is a CNF on $V = \{p, q, r\}$.

Description

This slide defines Conjunctive Normal Form (CNF) in the context of propositional logic. It states that a literal is either a propositional variable or the negation of a propositional variable. A clause is a disjunction (OR) of literals. A formula is in CNF if it is a conjunction (AND) of clauses. The slide provides an example of a formula in CNF, $(p \vee \neg q \vee r) \wedge (q \vee r) \wedge (\neg p \vee \neg q) \wedge r$, and specifies the variables used in this example as $V = \{p, q, r\}$. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a lesson on knowledge representation and reasoning, specifically focusing on propositional logic and how formulas can be represented in a standardized form for easier processing and manipulation by automated reasoning systems.

Figures

Slide 17

Content

Convert Any Functions to CNF

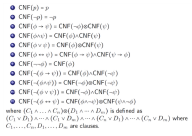
- $\text{CNF}(p) = p$
- $\text{CNF}(\neg p) = \neg p$
- $\text{CNF}(\phi \to \psi) = \text{CNF}(\neg\phi) \wedge \text{CNF}(\psi)$
- $\text{CNF}(\phi \wedge \psi) = \text{CNF}(\phi) \wedge \text{CNF}(\psi)$
- $\text{CNF}(\phi \vee \psi) = \text{CNF}(\phi) \vee \text{CNF}(\psi)$
- $\text{CNF}(\phi \leftrightarrow \psi) = \text{CNF}(\phi \to \psi) \wedge \text{CNF}(\psi \to \phi)$
- $\text{CNF}(\neg(\phi) = \text{CNF}(\phi)$
- $\text{CNF}(\neg(\phi \to \psi) = \text{CNF}(\phi) \wedge \text{CNF}(\neg\psi)$
- $\text{CNF}(\neg(\phi \wedge \psi) = \text{CNF}(\neg\phi) \vee \text{CNF}(\neg\psi)$
- $\text{CNF}(\neg(\phi \vee \psi) = \text{CNF}(\neg\phi) \wedge \text{CNF}(\neg\psi)$

where $(C_1 \wedge \cdots \wedge C_n)(D_1 \wedge \cdots \wedge D_m)$ is defined as $(C_1 \vee D_1) \wedge \cdots \wedge (C_n \vee D_m)$ where $C_1, \ldots, C_n, D_1, \ldots, D_m$ are clauses.

Description

This slide presents a list of rules for converting propositional logic formulas into Conjunctive Normal Form (CNF). Each numbered item shows a transformation of a specific logical connective or expression into its equivalent CNF form. For example, the rule for implication ($\phi \to \psi$) shows that it is equivalent to the conjunction of the negation of ϕ and ψ. The rules cover negation, conjunction, disjunction, implication, and biconditional. The final line defines how a conjunction of clauses (CNF) is combined with another conjunction of clauses. The context of the course (Artificial Intelligence Basics) indicates that this slide is part of a lesson on propositional logic, specifically focusing on normal forms and how to transform logical expressions into a standardized form for easier processing and manipulation by automated reasoning systems. The notation used is standard in propositional logic, with $\wedge$ representing AND, $\vee$ representing OR, $\to$ representing implication, and $\neg$ representing negation.

Figures



(a)

Slide 18

Content

Conversion to CNF

$B \leftrightarrow (P \vee Q)$

Eliminate $\leftrightarrow$, replacing $\alpha \leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$

- $(B \Rightarrow (P \vee Q)) \wedge ((P \vee Q) \Rightarrow B)$

  Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

- $(\neg B \vee P \vee Q) \wedge (\neg(P \vee Q) \vee B)$

  Move $\neg$ inwards using de Morgan's rules and double-negation:

- $(\neg B \vee P \vee Q) \wedge ((\neg P \wedge \neg Q) \vee B)$

  Apply distributivity law ($\wedge$ over $\vee$) and flatten:

- $(\neg B \vee P \vee Q) \wedge (\neg P \vee B) \wedge (\neg Q \vee B)$

Description

This slide shows the steps for converting a propositional logic formula into Conjunctive Normal Form (CNF). The starting formula is $B \leftrightarrow (P \vee Q)$. The first step eliminates the biconditional ($\leftrightarrow$) connective, replacing it with an equivalent conjunction of implications. Next, the implications are converted to disjunctions using the equivalence $\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$. Then, the negation is pushed inwards using De Morgan's laws and the double negation law. Finally, the conjunction is distributed over the disjunctions, resulting in a conjunction of clauses, which is the CNF form of the original formula. The steps are presented in a bulleted list, showing the transformations at each stage. The context of the course (Artificial Intelligence Basics) indicates that this slide is part of a lesson on propositional logic and automated reasoning, focusing on the process of converting logical expressions into a standard form for easier manipulation and processing by automated theorem provers.

Figures

Slide 19

Content

The Satisfiability Problem

Let V be a finite set of Boolean valued variables and let ϕ denote a CNF formula on V. An assignment v on V is a satisfying assignment for ϕ if we have $v(\phi) = \text{true}$. The propositional formula ϕ is satisfiable if there exists a satisfying assignment for ϕ. Deciding whether or not a propositional formula is satisfiable is called the Boolean satisfiability problem, denoted by SAT.

Description

This slide introduces the Satisfiability Problem (SAT) in the context of Artificial Intelligence Basics. It defines a satisfying assignment for a Conjunctive Normal Form (CNF) formula ϕ over a set of Boolean variables V. An assignment v is satisfying if applying the assignment to the formula results in a true value. The slide then defines a satisfiable formula as one for which a satisfying assignment exists. Finally, it states that the problem of determining whether a propositional formula is satisfiable is known as the Boolean satisfiability problem, denoted by SAT. This is a fundamental problem in computer science and artificial intelligence, with applications in various areas, including automated reasoning, knowledge representation, and constraint satisfaction.

Figures

Slide 20

Content

Applications of SAT (1)
Model Checking (hardware/software verification)

- Bounded model checking (BMC)
- Enhancement techniques: Abstraction and refinement
- Major groups at Intel, IBM, Microsoft, and universities such as CMU, Cornell, and Princeton
- SAT has become a dominant back-end technology

Key questions:

- Safety property: can system ever reach state S?
- Liveness property: will system always reach T after it gets to S?

Main idea:

- Encode "reachability" in a finite transition graph
- CNF encoding: in simple terms, "similar" to planning...

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses applications of the Satisfiability Problem (SAT). It focuses on Model Checking, a technique used for hardware and software verification. The slide highlights Bounded Model Checking (BMC) as a key application. It also mentions enhancement techniques like abstraction and refinement, and the significant role of SAT in this area, noting that it has become a dominant back-end technology. The slide then presents key questions related to safety and liveness properties in the context of system behavior. A safety property asks if a system can ever reach a specific state (S), while a liveness property asks if a system will always reach a target state (T) after reaching a starting state (S). Finally, the slide describes the main idea behind using SAT for model checking, which involves encoding reachability in a finite transition graph and using CNF encoding, drawing an analogy to planning. The inclusion of a circuit diagram image suggests a focus on hardware verification aspects.

Figures

(a)

Slide 21

Content

Applications of SAT (2)
Classical Planning

- Parallel step optimal plans
- E.g., SATPLAN-06 fastest in this category at IPC-06 planning competition

Main idea:

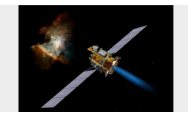
- Create planning graph, unfolded up to T steps
- Encode legal transitions as CNF
- Variables: action vars, fact vars for each time step t
- Clauses encode action preconditions, effects, mutual exclusion, etc.
- e.g., $(actionA_{(t+1)} \Rightarrow preA1_t \wedge preA2_t \wedge preA3_t)$,
- $(NOT\ actionA_t \vee NOT\ actionB_t)$, ...

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses applications of the Satisfiability Problem (SAT) in classical planning. It highlights the speed of the SATPLAN-06 algorithm in the IPC-06 planning competition. The main idea is to create a planning graph, unfolding it up to a certain number of steps (T). Legal transitions are encoded as Conjunctive Normal Form (CNF) formulas. Variables are introduced for actions and facts at each time step t. Clauses are used to represent preconditions, effects, and mutual exclusion constraints for actions. The example shows how actions and their preconditions are encoded using implications ($\Rightarrow$) and conjunctions ($\wedge$). The example also shows how mutual exclusion is encoded using disjunctions ($\vee$) and negations (NOT). The context of the course is focused on how SAT solvers can be used to find optimal plans in classical planning problems.

Figures

(a)



(b)

Slide 22

Content

Applications of SAT (3)
Combinatorial Design

- Complex mathematical structures with desirable properties
- Latin squares
- Quasi groups
- Ramsey numbers
- Steiner systems

Highly useful for

- design of experiments
- coding theory, cryptography
- drug design and testing
- crop rotation schedules

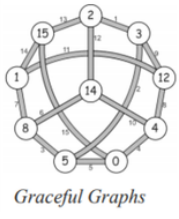
And more...

Description

This slide, part of a lecture on Artificial Intelligence Basics, details further applications of the Satisfiability Problem (SAT) focusing on Combinatorial Design. It lists various complex mathematical structures, such as Latin squares, quasi-groups, Ramsey numbers, and Steiner systems, that exhibit desirable properties. The slide emphasizes that these structures are highly useful in diverse fields, including the design of experiments, coding theory, cryptography, drug design and testing, and crop rotation scheduling. The slide also indicates that there are additional applications beyond those explicitly listed. The images on the slide (Graceful Graphs, Round-Robin Tournament, and Magic Squares) visually represent examples of combinatorial structures, but the primary focus of the slide is on the *applications* of SAT in the context of these structures, not the structures themselves.

Figures

(a)



(b)

(c)

(d)

Slide 23

Content

Applications of SAT (4)
Has been applied to solve sub-problems in many domains!

- Test pattern generation
- Scheduling
- Optimal Control
- Protocol Design (e.g., for networks)
- Multi-agent systems
- E-Commerce (E-auctions and electronic trading agents), etc.

Description

This slide, part of a lecture on Artificial Intelligence Basics, lists additional applications of the Satisfiability Problem (SAT). It states that SAT has been applied to solve sub-problems in various domains. The bullet points detail specific applications, including test pattern generation, scheduling, optimal control, protocol design (for example, in network contexts), multi-agent systems, and e-commerce (specifically mentioning e-auctions and electronic trading agents). The "etc." at the end implies that there are further applications not explicitly listed. The context of the course is focused on the broad range of problem-solving capabilities enabled by SAT algorithms.

Figures

Slide 24

Content

Satisfiability $\Longrightarrow$ 3-Satisfiability

To reduce the unrestricted SAT problem to 3-SAT, transform each clause $l_1 \vee \cdots \vee l_n$ to a conjunction of $n-2$ clauses

- $(l_1 \vee l_2 \vee x_2) \wedge$
- $(\neg x_2 \vee l_3 \vee x_3) \wedge$
- $(\neg x_3 \vee l_4 \vee x_4) \wedge \cdots \wedge$
- $(\neg x_{n-3} \vee l_{n-2} \vee x_{n-2}) \wedge$
- $(\neg x_{n-2} \vee l_{n-1} \vee l_n)$

where $x_2, \ldots, x_{n-2}$ are fresh variables not occurring elsewhere.

Although the two formulas are not logically equivalent, they are equisatisfiable.
https://en.wikipedia.org/wiki/Boolean_satisfiability_problem

Description

This slide discusses the reduction of the general Satisfiability problem (SAT) to the 3-Satisfiability problem (3-SAT). It describes a transformation process. The general clause $l_1 \vee l_2 \vee \cdots \vee l_n$ is converted into a conjunction of $n-2$ clauses. Each of these new clauses has exactly three literals. The new clauses introduce fresh variables $x_2, \ldots, x_{n-2}$ that do not appear elsewhere in the formula. The slide highlights that while the original and transformed formulas are not logically equivalent, they are equisatisfiable. This means that a satisfying assignment exists for the original formula if and only if a satisfying assignment exists for the transformed 3-SAT formula. The URL at the bottom of the slide links to the Wikipedia page on the Boolean satisfiability problem, providing further context and details on the topic.

Figures